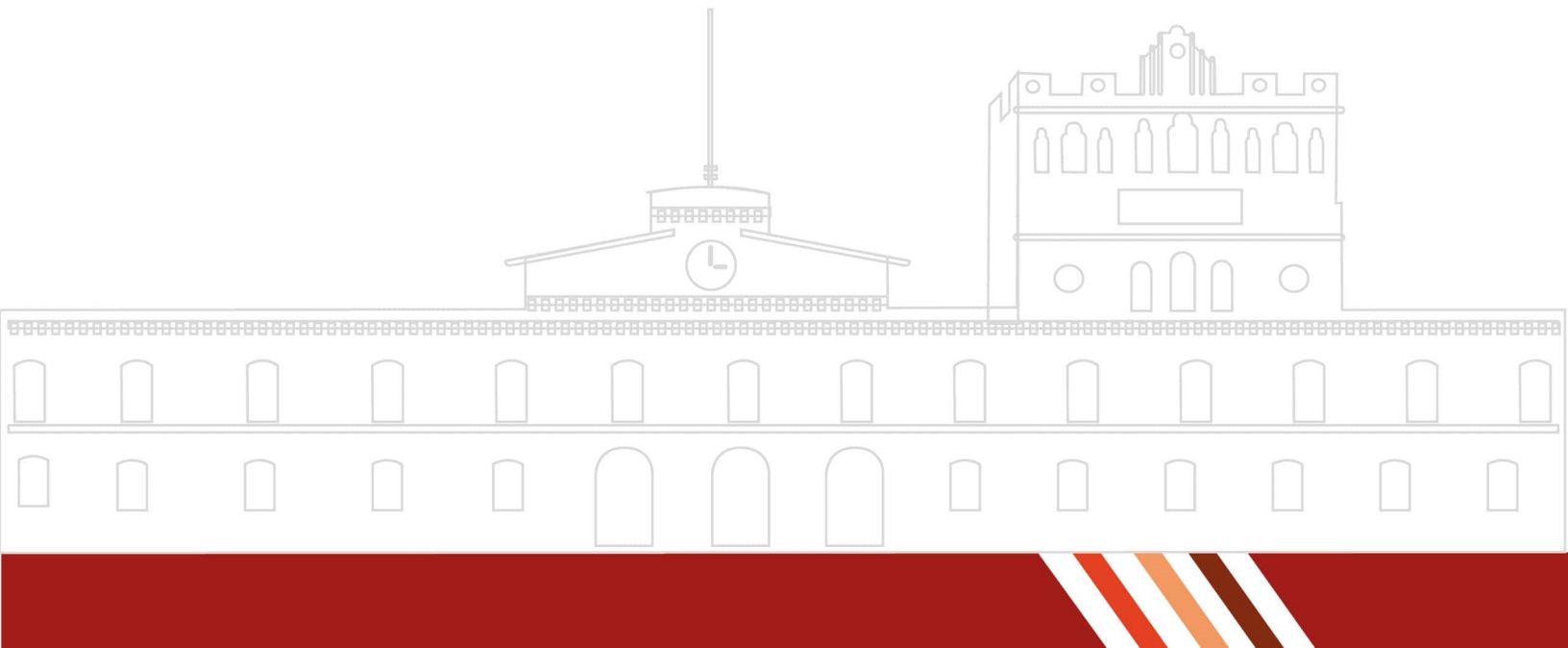


REPORTE DE PRÁCTICA NO. 2

**RE4CONOCIMIENTO DE PALABRAS CON AUTOMATAS
FINITOS DETERMINISTAS**

ALUMNO: JAVIER ALEJANDRO PINEDA FLORES

Dr. Eduardo Cornejo-Velázquez



1. Introducción

La teoría de autómatas y lenguajes formales constituye uno de los pilares conceptuales de las ciencias computacionales, ya que proporciona los fundamentos matemáticos sobre los que se construyen los compiladores, los sistemas de reconocimiento de patrones y numerosas técnicas empleadas dentro de la Inteligencia Artificial para el procesamiento simbólico y el reconocimiento de secuencias. Dentro de esta teoría, los Autómatas Finitos Deterministas (AFD) representan el modelo computacional más simple capaz de reconocer los lenguajes regulares, es decir, aquellos conjuntos de cadenas que pueden describirse mediante expresiones regulares o gramáticas de tipo 3 según la jerarquía de Chomsky. En la presente práctica se aborda el reconocimiento de palabras pertenecientes a distintos lenguajes definidos sobre diferentes alfabetos, mediante la construcción formal de Autómatas Finitos Deterministas. Para cada uno de los lenguajes propuestos se determina la tupla formal del autómata, su tabla de transición de estados y su diagrama de transición de estados, el cual es posteriormente construido y validado mediante simulación en el sitio Automaton Simulator (<https://automatonsimulator.com/>). Este proceso permite comprobar de manera práctica que el autómata diseñado acepta únicamente las palabras que pertenecen al lenguaje L y rechaza aquellas que no le pertenecen. El presente documento se organiza en las secciones establecidas en el formato de reporte del manual de prácticas: introducción, marco teórico, objetivos, desarrollo, resultados, conclusiones y bibliografía. En la sección de desarrollo se incluyen los espacios correspondientes para que el estudiante complete, para cada uno de los diez ejercicios propuestos, la tupla del AFD, la tabla de transición de estados, la captura del diagrama de transición obtenido en el simulador y las palabras aceptadas y rechazadas utilizadas para validar cada autómata.

2. Marco teórico

Autómatas Finitos Deterministas

Un **Autómata Finito Determinista (AFD)** se define formalmente como una tupla de cinco elementos:

$$A = (\Sigma, Q, \delta, q_0, F)$$

donde:

- Σ : es el **alfabeto de entrada**, es decir, el conjunto finito de símbolos válidos.
- Q : es el conjunto finito y no vacío de estados del autómata.
- δ : es la **función de transición**,

$$\delta : Q \times \Sigma \rightarrow Q,$$

que determina el estado al que se pasa dado un estado actual y un símbolo de entrada.

- $q_0 \in Q$: es el **estado inicial** del autómata.
- $F \subseteq Q$: es el conjunto de **estados finales o de aceptación**, con $F \neq \emptyset$.

Sea $A = (\Sigma, Q, \delta, q_0, F)$ un AFD y sea

$$w = w_1 w_2 \cdots w_n$$

una cadena de símbolos donde $w_i \in \Sigma$. Se dice que A **acepta** w si existe una secuencia de estados

$$r_0, r_1, \dots, r_n \in Q$$

que cumple las siguientes tres condiciones:

1. $r_0 = q_0$, es decir, el autómata inicia en el estado inicial.
- 2.

$$\delta(r_i, w_{i+1}) = r_{i+1}, \quad i = 0, 1, \dots, n-1,$$

es decir, el autómata cambia de estado conforme a la función de transición.

3. $r_n \in F$, es decir, la cadena termina en un estado de aceptación.

En consecuencia, A reconoce el lenguaje L si

$$L = \{w \mid A \text{ acepta } w\}.$$

Tabla de transición de estados

Una **tabla de transición de estados** es una representación tabular de la función δ . Muestra, para cada estado del autómata y cada símbolo del alfabeto de entrada, cuál es el estado siguiente al que se transita. En esencia, es una tabla de verdad en la que las entradas son el estado actual y el símbolo leído, mientras que las salidas corresponden al estado resultante. Esta tabla constituye una herramienta indispensable para la implementación programática del autómata, ya que traduce de forma directa la definición formal de δ en una estructura de datos fácilmente codificable.

Diagrama de transición de estados

El **diagrama de transición de estados** es la representación gráfica del AFD. Cada nodo del diagrama corresponde a un estado. El estado inicial se indica mediante una flecha que no tiene nodo de origen, mientras que los estados finales se representan mediante un círculo doble. Si existe una transición del estado e_i al estado e_j para un símbolo de entrada a , entonces existe un arco rotulado con a que va del nodo e_i al nodo e_j . Formalmente:

$$\delta(e_i, a) = e_j.$$

Esta representación se ilustra en la Figura ??.

Simulación de AFD

La simulación de un AFD consiste en ejecutar, sobre una implementación (física o mediante software) del autómata diseñado, distintas cadenas de entrada para verificar si son aceptadas o rechazadas conforme a la definición del lenguaje L. En esta práctica la simulación se realiza mediante la herramienta en línea Automaton Simulator (<https://automatonsimulator.com/>), la cual permite construir el diagrama de transición de forma gráfica y ejecutar palabras de prueba paso a paso, mostrando el recorrido de estados y el resultado de aceptación o rechazo. Esta validación experimental complementa el diseño teórico del autómata y permite detectar errores en la definición de la función de transición o en la selección del conjunto de estados finales.

REFERENCIAS

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Educación.

REFERENCIAS

Kelley, D. (1995). *Teoría de autómatas y lenguajes formales*. Prentice Hall.

REFERENCIAS

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Educación.

REFERENCIAS

Elmasri, R., & Navathe, S. B. (2007). *Fundamentos de sistemas de bases de datos* (5.^a ed.). Pearson Educación.

3. Herramientas empleadas

En esta práctica la simulación se realiza mediante la herramienta en línea Automaton Simulator (<https://automatonsimulator.com>) la cual permite construir el diagrama de transición de forma gráfica y ejecutar palabras de prueba paso a paso, mostrando el recorrido de estados y el resultado de aceptación o rechazo.

4. Desarrollo

Para el desarrollo de la práctica se resolvieron los diez ejercicios propuestos en el manual. Para cada uno de ellos se determinó, sobre el alfabeto Σ indicado, un Autómata Finito Determinista capaz de reconocer el lenguaje L especificado. Para la resolución de cada ejercicio se siguió el siguiente procedimiento:

1. **Definición formal del AFD:** se estableció la tupla

$$A = (\Sigma, Q, \delta, q_0, F).$$

2. **Construcción de la tabla de transición de estados:** se determinaron las transiciones correspondientes a cada estado y símbolo del alfabeto de entrada, de acuerdo con la función

$$\delta : Q \times \Sigma \rightarrow Q.$$

3. **Construcción del diagrama de transición de estados:** se elaboró y simuló el diagrama correspondiente a cada autómata mediante el sitio web *Automaton Simulator*.
4. **Validación del autómata:** se comprobó el funcionamiento de cada AFD mediante la ejecución de cinco palabras aceptadas y cinco palabras rechazadas por el lenguaje L .

EJERCICIO 1

Alfabeto:

$$\Sigma = \{0, 1\}$$

Lenguaje:

$$L = \{ 0x \mid x \in \{0, 1\}^* \}$$

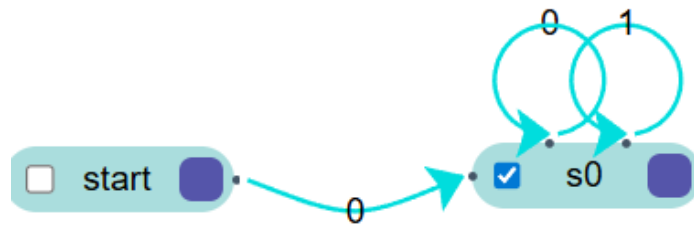
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, 0)$	$\delta(\cdot, 1)$
$*q_0$	s_0	—
$**s_0$	s_0	s_0

Diagrama de transición de estados



Expresión regular

$$r = 0(0 \mid 1)^*$$

EJERCICIO 2

Alfabeto:

$$\Sigma = \{0, 1\}$$

Lenguaje:

$$L = \{ x1 \mid x \in \{0, 1\}^+ \}$$

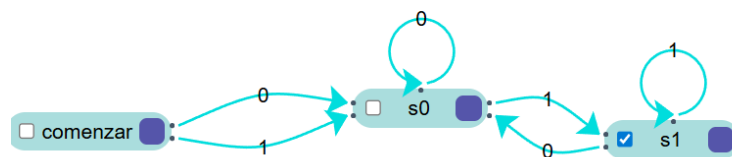
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, 0)$	$\delta(\cdot, 1)$
$*q_0$	s_0	s_0
s_0	s_0	s_1
$**s_1$	s_0	s_1

Diagrama de transición de estados



Expresión regular

$$r = (0 \mid 1)(0 \mid 1)^*1$$

EJERCICIO 3

Alfabeto:

$$\Sigma = \{0, 1\}$$

Lenguaje:

$$L = \{x01y \mid x, y \in \{0, 1\}^*\}$$

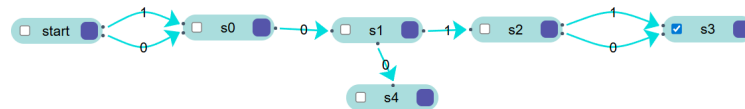
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, 0)$	$\delta(\cdot, 1)$
$*q_0$	s_0	s_0
s_0	s_1	—
s_1	s_4	s_2
s_2	s_3	s_3
$**s_3$	—	—
s_4	—	—

Diagrama de transición de estados



Test Results:

- Accept: 1010 -- Pass
- Accept: 0010 -- Pass
- Accept: 1011 -- Pass
- Accept: 1011 -- Pass
- Reject: 1111 -- Pass
- Reject: 1101 -- Pass
- Reject: 1110 -- Pass
- Reject: [Empty String] -- Pass
- Reject: 0000 -- Pass
- Reject: [Empty String] -- Pass

Expresión regular

$$r = (0 \mid 1)^*01(0 \mid 1)^*$$

EJERCICIO 4

Alfabeto:

$$\Sigma = \{0, 1\}$$

Lenguaje:

$$L = \{x110y \mid x, y \in \{0, 1\}^*\}$$

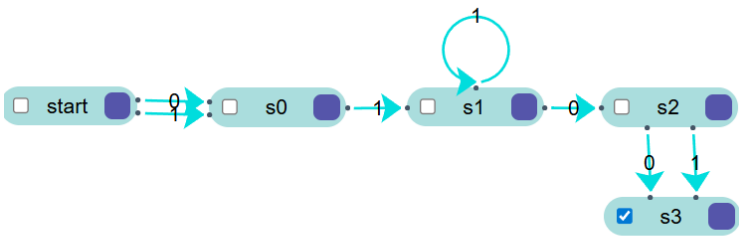
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, 0)$	$\delta(\cdot, 1)$
$*q_0$	s_0	s_0
s_0	—	s_1
s_1	s_2	s_1
s_2	s_3	s_3
$**s_3$	—	—

Diagrama de transición de estados



Test Results:
Accept: 11101 -- Pass
Accept: 01100 -- Pass
Accept: 01101 -- Pass
Accept: 11100 -- Pass
Reject: 00000 -- Pass
Reject: 11111 -- Pass
Reject: 11110 -- Pass
Reject: 00011 -- Pass

Expresion regular

$$r = (0 \mid 1)^* 110 (0 \mid 1)^*$$

EJERCICIO 5

Alfabeto:

$$\Sigma = \{a, b, c\}$$

Lenguaje:

$$L = \{acxab \mid x \in \{a, b, c\}^*\}$$

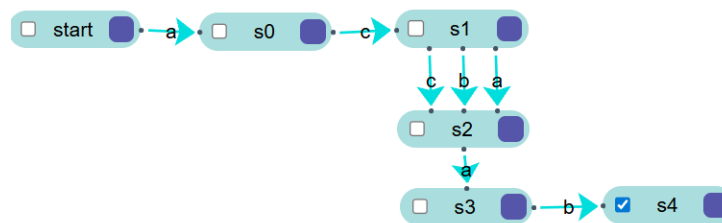
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, a)$	$\delta(\cdot, b)$	$\delta(\cdot, c)$
$*q_0$	s_0	—	—
s_0	—	—	s_1
s_1	s_2	s_2	s_2
s_2	s_3	—	—
s_3	—	s_4	—
$**s_4$	—	—	—

Diagrama de transición de estados



Test Results:

Accept: acaab -- Pass
 Accept: acbab -- Pass
 Accept: accab -- Pass
 Reject: abcca -- Pass
 Reject: acaac -- Pass
 Reject: abbab -- Pass
 Reject: aaaaa -- Pass
 Reject: bbbbb -- Pass
 Reject: ccccc -- Pass

Expresión regular

$$r = ac(a \mid b \mid c)^*ab$$

EJERCICIO 6

Alfabeto:

$$\Sigma = \{a, b, c\}$$

Lenguaje:

$$L = \{acxz \mid x \in \{a, b, c\}^*, z \neq ab\}$$

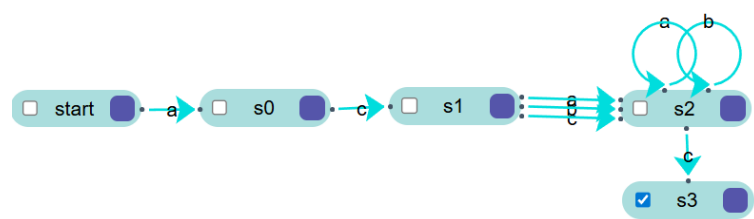
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, a)$	$\delta(\cdot, b)$	$\delta(\cdot, c)$
$*q_0$	s_0	—	—
s_0	—	—	s_1
s_1	s_2	s_2	s_2
s_2	s_2	s_2	s_3
$**s_3$	—	—	—

Diagrama de transición de estados



Test Results:	
Accept: acac	Pass
Accept: acbc	Pass
Accept: accc	Pass
Reject: acaa	Pass
Reject: acba	Pass
Reject: acca	Pass
Reject: acaa	Pass
Reject: acab	Pass
Reject: acca	Pass
Reject: accb	Pass

Expresión regular

$r = ac(a \mid b \mid c)(a \mid b)^*c$

EJERCICIO 7

Alfabeto:

$\Sigma = \{a, b, c, d\}$

Lenguaje:

$L = \{ acbxz \mid x \in \{a, b, c, d\}^*, \; z \neq bd \}$

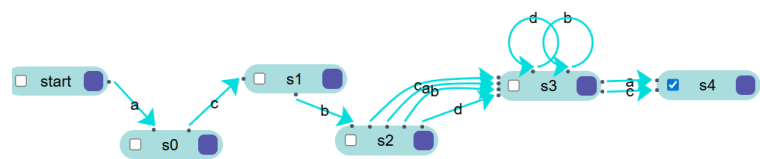
Tupla del AFD:

$A = (\Sigma, Q, \delta, q_0, F)$

Tabla de transición de estados

Estado	$\delta(\cdot, a)$	$\delta(\cdot, b)$	$\delta(\cdot, c)$	$\delta(\cdot, d)$
$*q_0$	s_0	—	—	—
s_0	—	—	s_1	—
s_1	—	s_2	—	—
s_2	s_3	s_3	s_3	s_3
s_3	s_4	s_3	s_4	s_3
$* \; * \; s_4$	—	—	—	—

Diagrama de transición de estados



est Results:	
Accept: acbaa	Pass
Accept: acbba	Pass
Accept: acbca	Pass
Accept: acbac	Pass
Accept: acbbc	Pass
Accept: acbcc	Pass
Reject: acaaa	Pass
Reject: acbbb	Pass
Reject: accaaa	Pass
Reject: acaab	Pass
Reject: acbcb	Pass

Expresión regular

$$r = acb(a \mid b \mid c \mid d)(b \mid d)^*(a \mid c)$$

EJERCICIO 8

Alfabeto:

$$\Sigma = \{a, b, c, d\}$$

Lenguaje:

$$L = \{ cabaxz \mid x \in \{a, b, c, d\}^*, z \neq ab \}$$

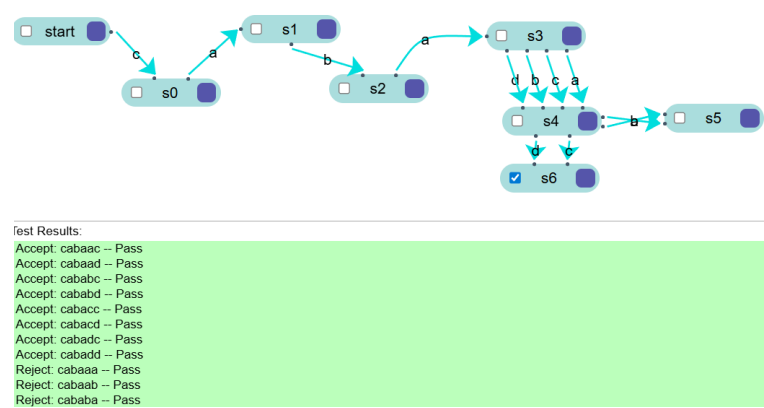
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, a)$	$\delta(\cdot, b)$	$\delta(\cdot, c)$	$\delta(\cdot, d)$
$*q_0$	—	—	s_0	—
s_0	s_1	—	—	—
s_1	—	s_2	—	—
s_2	s_3	—	—	—
s_3	s_4	s_4	s_4	s_4
s_4	—	s_5	s_6	s_6
s_5	—	—	—	—
$**s_6$	—	—	—	—

Diagrama de transición de estados



Expresión regular

$$r = caba(a \mid b \mid c \mid d)(c \mid d)$$

EJERCICIO 9

Alfabeto:

$$\Sigma = \{a, b, c\}$$

Lenguaje:

$$L = \{ a^n cb^m \mid n > 0, m > 0 \}$$

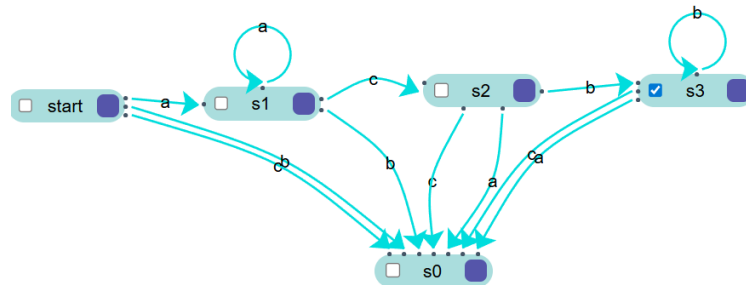
Tupla del AFD:

$$A = (\Sigma, Q, \delta, q_0, F)$$

Tabla de transición de estados

Estado	$\delta(\cdot, a)$	$\delta(\cdot, b)$	$\delta(\cdot, c)$
<i>*start</i>	<i>s</i> ₁	<i>s</i> ₀	<i>s</i> ₀
<i>s</i> ₁	<i>s</i> ₁	<i>s</i> ₀	<i>s</i> ₂
<i>s</i> ₂	<i>s</i> ₀	<i>s</i> ₃	<i>s</i> ₀
<i>* * s</i> ₃	<i>s</i> ₀	<i>s</i> ₃	<i>s</i> ₀
<i>s</i> ₀	<i>s</i> ₀	<i>s</i> ₀	<i>s</i> ₀

Diagrama de transición de estados



Test Results:

Accept: acb -- Pass
 Accept: aacb -- Pass
 Accept: acbb -- Pass
 Accept: aaacbbb -- Pass
 Accept: aaaacb -- Pass
 Reject: cb -- Pass
 Reject: ab -- Pass
 Reject: aabb -- Pass
 Reject: acbc -- Pass

Expresión regular

$$r = a a^* c b b^*$$

EJERCICIO 10

Alfabeto:

$$\Sigma = \{a, b, c\}$$

Lenguaje:

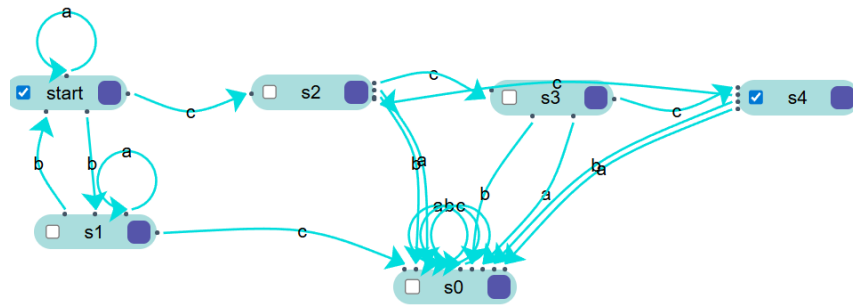
$$L = \{x c^{3m} \mid x \in \{a, b\}^*, \text{ la cantidad de } b' \text{ en } x \text{ es par}, m \geq 0\}$$

Tupla del AFD: $A = (\Sigma, Q, \delta, q_0, F)$

Tabla de transición de estados

Estado	$\delta(\cdot, a)$	$\delta(\cdot, b)$	$\delta(\cdot, c)$
**start	start	s ₁	s ₂
s ₁	s ₁	start	s ₀
s ₂	s ₀	s ₀	s ₃
s ₃	s ₀	s ₀	s ₄
* * s ₄	s ₀	s ₀	s ₂
s ₀	s ₀	s ₀	s ₀

Diagrama de transición de estados



Test Results:

Accept: bb -- Pass
Accept: aabb -- Pass
Accept: ccc -- Pass
Accept: abccc -- Pass
Accept: bbccccc -- Pass
Reject: b -- Pass
Reject: abcc -- Pass
Reject: aabbc -- Pass
Reject: bbcccc -- Pass
Reject: bcab -- Pass

Expresión regular

$$r = (a^*ba^*b)^*a^*(ccc)^*$$

CUESTIONARIO

1. ¿Cuáles son los elementos que definen un AFD?

Un AFD se define formalmente como la tupla $(\Sigma, Q, \delta, q_0, F)$: Σ es el alfabeto de entrada (símbolos válidos); Q es el conjunto finito de estados; δ es la función de transición que indica a qué estado se pasa dado un estado actual y un símbolo; q_0 es el estado inicial; y F es el conjunto de estados de aceptación.

2. ¿Cuál es la utilidad de una tabla de transiciones de estado de un AFD?

Representa de forma ordenada y completa la función δ , mostrando para cada estado y cada símbolo del alfabeto cuál es el estado siguiente. Esto facilita verificar que el autómata esté bien definido (que no falte ninguna transición), sirve como base directa para programar el autómata, y permite trazar manualmente el recorrido de cualquier cadena sin necesidad del diagrama.

3. ¿Qué importancia tienen los diagramas de transiciones de estado en el proceso de construcción de un AFD?

Permiten visualizar de forma clara e intuitiva el comportamiento completo del autómata: qué estado es el inicial, cuáles son de aceptación y cómo se conectan mediante las transiciones. Esto ayuda a detectar errores de diseño (estados sin transición, ciclos mal definidos, estados de aceptación incorrectos) antes de implementarlo, y facilita comunicar y comprobar el diseño con el simulador.

4. ¿Cuáles son las ventajas de la simulación del AFD?

Permite validar experimentalmente que el autómata diseñado reconoce correctamente el lenguaje deseado, probando cadenas aceptadas y rechazadas sin necesidad de escribir código. Facilita detectar errores de construcción de forma inmediata, y sirve como herramienta de aprendizaje para comprender de manera práctica el funcionamiento de la teoría de autómatas.

CONCLUSIONES

La realización de esta práctica permitió aplicar de manera integral los conceptos fundamentales de la teoría de autómatas finitos deterministas: la definición formal de la tupla $(\Sigma, Q, \delta, q_0, F)$, la construcción de tablas de transición de estados y su correspondiente representación gráfica mediante diagramas de transición. Se comprobó que estas tres representaciones —formal, tabular y gráfica— son equivalentes entre sí y ofrecen distintas ventajas: la definición formal facilita el análisis matemático del autómata, la tabla de transición favorece su implementación computacional, y el diagrama de transición permite una comprensión visual e intuitiva del comportamiento del sistema.

Asimismo, la simulación de los autómatas construidos en la herramienta Automaton Simulator resultó fundamental para validar experimentalmente el diseño teórico de cada AFD, permitiendo verificar de forma inmediata si una palabra es aceptada o rechazada y facilitando la identificación de errores en la definición de estados o transiciones antes de dar por concluido el diseño. De esta manera, se cumplieron los objetivos planteados, logrando construir autómatas finitos deterministas capaces de reconocer correctamente los lenguajes regulares definidos en cada uno de los diez ejercicios propuestos.

REFERENCIAS

- Brookshear, J. G. (1995). *Teoría de la computación: Lenguajes formales, autómatas y complejidad*. Addison-Wesley Iberoamericana.
- Giró, J., Vázquez, J., Meloni, B., y Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Alfaomega.
- Ruiz Catalán, J. (2010). *Compiladores: Teoría e implementación*. Alfaomega.